Programare Web / Dezvoltarea Aplicațiilor Web
Tehnologia Informației, anul IV · Informatică, anul II · Anul universitar 2026–2027, semestrul I

LABORATORUL 03

JavaScript: DOM, evenimente și comunicare asincronă (fetch)

Facem pagina să reacționeze la acțiunile utilizatorului, apoi cerem date reale de la un API public și le afișăm, tratând cu grijă așteptarea și erorile.

Tema	Temele 5, 6 și 11 din planul disciplinei (examene.pe.ub.ro): JavaScript, DOM și evenimente, comunicarea asincronă
Durata	2 ore
Aveți nevoie de	proiectul `daw-template-ti-info-2026-2027` (laboratoarele 01–02), editorul (VS Code cu Live Server sau PhpStorm/WebStorm), un browser și conexiune la internet
La final aveți	folderul `exercitii/lab03/` cu o pagină interactivă care afișează vremea din Bacău, salvat pe GitHub

1. Obiective

La finalul laboratorului veți putea:

- selecta și modifica elemente din pagină prin DOM;
- reacționa la evenimente (click, tastare, trimiterea unui formular);
- valida simplu datele dintr-un formular și afișa mesaje clare;
- cere date de la un API cu `fetch` și `async/await` și le afișa;
- trata așteptarea („Se încarcă…”) și erorile, fără ca pagina să se blocheze.

2. Pe scurt: DOM, evenimente, JSON și fetch

Când încarcă o pagină, browserul construiește din HTML un arbore de obiecte numit **DOM** (*Document Object Model*). JavaScript poate citi și modifica acest arbore: schimbă texte, adaugă elemente, pune sau scoate clase CSS [1], [2]. Codul nu rulează de la sine, ci ca răspuns la **evenimente**: un click, o tastă, trimiterea unui formular.

Aplicațiile moderne cer date de la server **fără să reîncarce pagina**. Funcția `fetch` trimite o cerere HTTP, iar `await` așteaptă răspunsul fără să blocheze interfața. Datele vin de obicei în format **JSON** (*JavaScript Object Notation*): perechi `"cheie": valoare`, grupate în obiecte `{ }` și liste `[ ]`. Exact așa comunică frontend-ul React cu backend-ul Symfony din template-ul nostru [3], [4].

3. Pornirea

1. Porniți **Docker Desktop**. Deschideți folderul `daw-template-ti-info-2026-2027` în editor și terminalul integrat.
2. Verificați pe ce branch sunteți: `git branch` marchează ramura curentă cu `*` (sau `git status`, pe primul rând: `On branch …`). Doar dacă nu sunteți deja pe branch-ul vostru, treceți pe el; apoi porniți containerele (înlocuiți `popescu.ion`):

```
git branch                # ramura curenta are * in fata
# doar daca NU sunteti deja pe branch-ul vostru:
git checkout popescu.ion   # sau: git switch popescu.ion
docker compose -f backend/compose.yaml up -d
docker compose -f frontend/compose.yaml up -d
```

3.1 De ce lucrăm în afara aplicației React

Aplicația React din `frontend/` este construită tot din HTML, CSS și JavaScript, dar le ascunde în spatele unor noțiuni noi: componente, JSX, stare. Ca să înțelegeți ce face React în locul vostru, scrieți întâi aceleași lucruri „de mână”, în pagini simple, fără framework. Trecerea se face treptat:

Laboratorul	Ce învățăm	Unde lucrăm
02	structura (HTML5) și aspectul (CSS3) unei pagini	`exercitii/lab02/`
03	comportamentul (JavaScript): DOM, evenimente, date de la un API	`exercitii/lab03/`
04 și următoarele	aceleași idei, organizate în componente React	`frontend/app/src/`

Paginile de exercițiu stau separat de aplicație, ca să nu o modifice și să nu o strice, dar în același repository, pe branch-ul vostru, ca să fie salvate pe GitHub împreună cu restul lucrului.

3.2 Unde creați fișierele

În **rădăcina proiectului** (folderul principal, lângă `backend`, `frontend` și `postman`) creați folderul `exercitii` (dacă nu există deja de la laboratorul 02), iar în el subfolderul `lab03`, cu fișierele `index.html`, `style.css` și `script.js`:

```
daw-template-ti-info-2026-2027/
├── backend/
├── frontend/
├── postman/
└── exercitii/
    ├── lab02/    index.html, style.css
    └── lab03/    index.html, style.css, script.js
```

Folderele se creează cu click dreapta în panoul de fișiere al editorului (**New Folder** / **New → Directory**) sau din terminalul integrat, care este deja în rădăcina proiectului: `mkdir -p exercitii/lab03`.

3.3 Cum deschideți pagina în browser

Paginile din `exercitii/` sunt fișiere statice: nu au nevoie de Docker, ci doar de un mic server web local, care le deschide în browser și le reîncarcă după fiecare salvare.

Editor	Cum deschideți pagina	Adresa din browser
VS Code	instalați o singură dată extensia **Live Server** (Extensions → căutați „Live Server”, autor Ritwick Dey → Install); apoi click dreapta pe `index.html` → **Open with Live Server** (sau butonul **Go Live** din bara de jos). Pagina se reîncarcă singură la fiecare salvare.	`http://127.0.0.1:5500/exercitii/lab03/index.html`
PhpStorm / WebStorm	au serverul integrat, fără extensie: deschideți `index.html` și duceți mouse-ul în colțul din dreapta-sus al editorului, unde apar pictogramele browserelor, apoi alegeți browserul (sau click dreapta pe fișier → **Open In → Browser**). Pentru reîncărcare automată: **Settings → Tools → Web Browsers and Preview → Reload page in browser: On Save**.	`http://localhost:63342/daw-template-ti-info-2026-2027/exercitii/lab03/index.html`

> **Sfat.** Dacă pagina nu se actualizează, salvați fișierul (**Ctrl + S**, pe macOS **Cmd + S**) și reîncărcați manual (**F5**). Deschiderea cu dublu-click pe fișier funcționează și ea, dar fără reîncărcare automată.

4. DOM și evenimente

1. În `index.html` scrieți pagina. Scriptul se încarcă la sfârșitul lui `<body>`, după ce elementele există:

```
<!DOCTYPE html>
<html lang="ro">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Laboratorul 03 — JavaScript</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1 id="titlu">Laboratorul 03</h1>
  <button id="btn-tema" type="button">Schimbă tema</button>
  <h2>Formular</h2>
  <form id="formular" novalidate>
    <label for="nume">Numele vostru</label>
    <input id="nume" type="text" autocomplete="name">
    <button type="submit">Trimite</button>
    <p id="eroare-nume" class="eroare"></p>
  </form>
  <p>Previzualizare: Salut, <strong id="previzualizare">...</strong>!</p>
  <h2>Vremea în Bacău</h2>
  <button id="btn-vreme" type="button">Afișează vremea</button>
  <p id="stare"></p>
  <ul id="vreme"></ul>
  <script src="script.js"></script>
</body>
</html>
```

2. În `style.css`, două clase pe care le vom pune și scoate din JavaScript:

```
body { font-family: system-ui, Arial, sans-serif; margin: 2rem; }
.intunecat { background: #1f2937; color: #f9fafb; }
.eroare { color: #b42318; }
```

3. În `script.js`, selectați elementele și legați evenimentele:

```
// 1. Selectăm elementele o singură dată
const titlu = document.querySelector('#titlu');
const btnTema = document.querySelector('#btn-tema');
const formular = document.querySelector('#formular');
const inputNume = document.querySelector('#nume');
const eroareNume = document.querySelector('#eroare-nume');
const previzualizare = document.querySelector('#previzualizare');

// 2. Click: punem sau scoatem clasa „intunecat”
btnTema.addEventListener('click', () => {
  document.body.classList.toggle('intunecat');
});

// 3. Tastare: afișăm în timp real ce scrie utilizatorul
inputNume.addEventListener('input', () => {
  previzualizare.textContent = inputNume.value || '...';
});
```

4. Tot în `script.js`, tratați trimiterea formularului:

```
// 4. Trimitere: validăm înainte de a folosi datele
formular.addEventListener('submit', (event) => {
  event.preventDefault(); // oprim reîncărcarea paginii
  const nume = inputNume.value.trim();
  if (nume.length < 3) {
    eroareNume.textContent = 'Numele trebuie să aibă cel puțin 3 caractere.';
    inputNume.focus();
    return;
  }
  eroareNume.textContent = '';
  titlu.textContent = `Bine ai venit, ${nume}!`;
  formular.reset();
});
```

5. Deschideți pagina în browser, ca la secțiunea 3.3, și consola browserului (**F12 → Console**), ca să vedeți eventualele erori.

Rezultat. Butonul **Schimbă tema** comută fundalul închis. Ce tastați apare imediat în rândul „Previzualizare”. Cu un nume scurt, sub formular apare mesajul roșu; cu un nume valid, titlul devine „Bine ai venit, …!”.

De ce. `textContent` pune text simplu, nu cod HTML, așa că ce scrie utilizatorul nu poate injecta etichete în pagină.

5. Date de la un API: fetch, async/await, erori

Folosim **Open-Meteo**, un API meteo public, fără cheie de acces [5]. Deschideți întâi adresa API-ului pentru Bacău direct în browser. Parametrii din adresă cer coordonatele orașului (`latitude`, `longitude`) și valorile curente (`current`) pentru temperatură și vânt. Răspunsul JSON arată, prescurtat, astfel (valorile diferă de la o oră la alta):

```
{
  "latitude": 46.57,
  "longitude": 26.91,
  "current_units": { "temperature_2m": "°C", "wind_speed_10m": "km/h" },
  "current": {
    "time": "2026-10-07T12:00",
    "temperature_2m": 14.2,
    "wind_speed_10m": 9.4
  }
}
```

1. Adăugați la sfârșitul lui `script.js` adresa, elementele și o funcție ajutătoare:

```
const URL_VREME = 'https://api.open-meteo.com/v1/forecast'
  + '?latitude=46.57&longitude=26.91&current=temperature_2m,wind_speed_10m';
const btnVreme = document.querySelector('#btn-vreme');
const stare = document.querySelector('#stare');
const listaVreme = document.querySelector('#vreme');

function adaugaRand(text) {
  const li = document.createElement('li');
  li.textContent = text;
  listaVreme.append(li);
}
```

2. Apoi funcția asincronă care cere datele, le afișează și tratează erorile:

```
async function incarcaVremea() {
  stare.textContent = 'Se încarcă…';
  listaVreme.innerHTML = '';
  btnVreme.disabled = true;
  try {
    const raspuns = await fetch(URL_VREME);
    if (!raspuns.ok) {
      throw new Error('Serverul a răspuns cu codul ' + raspuns.status);
    }
    const date = await raspuns.json();
    const acum = date.current;
    const u = date.current_units;
    adaugaRand(`Temperatura: ${acum.temperature_2m} ${u.temperature_2m}`);
    adaugaRand(`Vântul: ${acum.wind_speed_10m} ${u.wind_speed_10m}`);
    adaugaRand(`Ora măsurătorii (GMT): ${acum.time}`);
    stare.textContent = '';
  } catch (eroare) {
    console.error(eroare);
    stare.textContent = 'Nu am putut încărca vremea. Încercați din nou.';
  } finally {
    btnVreme.disabled = false;
  }
}

btnVreme.addEventListener('click', incarcaVremea);
```

3. Apăsați **Afișează vremea**. Apoi, în F12 → **Network**, alegeți *Offline* din lista *No throttling* și apăsați din nou butonul, ca să vedeți tratarea erorii. Reveniți la *No throttling*.

Rezultat. Apare scurt „Se încarcă…”, apoi lista cu temperatura, vântul și ora. În modul *Offline* apare mesajul prietenos, iar detaliile tehnice ajung doar în consolă. Butonul este dezactivat cât timp cererea este în curs. Pe scurt: `await` așteaptă răspunsul (doar într-o funcție `async`), `raspuns.ok` este `true` pentru codurile 200–299, `raspuns.json()` transformă textul JSON în obiect, `catch` prinde orice eroare, iar `finally` rulează în orice caz.

6. Același mecanism în template-ul nostru

Deschideți http://localhost:8080/api/doc, apoi consola (**F12 → Console**) și rulați:

```
fetch('/api/products').then(r => r.json()).then(console.log)
```

Rezultat. În consolă apare lista produselor din baza de date, ca obiecte JavaScript (sau o listă goală, `[]`, dacă nu ați adăugat produse). Este exact apelul pe care îl face pagina **Produse** din aplicația React, prin funcțiile din `frontend/app/src/api.js`.

Sfat. Dacă Chrome refuză lipirea codului în consolă, tastați `allow pasting`, apăsați **Enter** și lipiți din nou.

7. Exerciții

1. Adăugați un buton **Resetează** care readuce titlul la „Laboratorul 03” și golește lista vremii.
2. Extindeți validarea: numele nu poate conține cifre (indiciu: `/[0-9]/.test(nume)`). Afișați un mesaj diferit pentru fiecare problemă.
3. Afișați, lângă temperatură, un cuvânt de tipul „rece” (sub 10 °C), „plăcut” (10–25 °C) sau „cald” (peste 25 °C).
4. ★ (opțional) Alegerea orașului dintr-o listă. Adăugați în HTML lista de mai jos, iar în `incarcaVremea` construiți adresa din valoarea aleasă (`select.value.split(',')` dă latitudinea și longitudinea):

```
<select id="oras">
  <option value="46.57,26.91">Bacău</option>
  <option value="47.16,27.59">Iași</option>
  <option value="44.43,26.10">București</option>
  <option value="46.77,23.60">Cluj-Napoca</option>
</select>
```

5. ★ (opțional) Adăugați în adresa API-ului parametrul `&timezone=auto`, ca ora să fie afișată în ora locală, și actualizați eticheta „(GMT)”.

8. Salvarea lucrului și oprirea

```
git status
git add .
git commit -m "Laboratorul 03: DOM, evenimente si vremea cu fetch"
git push origin popescu.ion
# sau doar: git push (dupa ce ati facut o data git push -u)
docker compose -f backend/compose.yaml stop
docker compose -f frontend/compose.yaml stop
```

Push. `git push origin popescu.ion` este forma completă: trimite branch-ul vostru la `origin`, adresa proiectului de pe GitHub. Forma scurtă, `git push`, merge doar dacă ați făcut anterior, o dată, `git push -u origin popescu.ion`; `-u` leagă branch-ul local de cel de pe GitHub.

9. Ce arătați la final

- pagina în funcțiune: tema, previzualizarea, validarea și vremea din Bacău;
- mesajul de eroare în modul *Offline* și rezultatul apelului din consolă pe `/api/doc`;
- exercițiile 1–3 rezolvate și commit-ul vizibil pe GitHub, pe branch-ul vostru.

Resurse pentru exersare

- **W3Schools – HTML DOM:** https://www.w3schools.com/js/js_htmldom.asp
- **W3Schools – evenimente:** https://www.w3schools.com/js/js_events.asp
- **W3Schools – Fetch API:** https://www.w3schools.com/js/js_api_fetch.asp
- **Curs JavaScript în limba română (playlist YouTube):** https://www.youtube.com/playlist?list=PL7aWL1nkGk0ytE-fV1pApIdKI2ZwIyw-5
- **Ghid JavaScript în română:** https://js.punctsivirgula.ro/
- **Open-Meteo – documentația API:** https://open-meteo.com/en/docs

Bibliografie

[1] E. Pruteanu și I. Furdu, *Modele, tehnologii, servicii și aplicații integrate de tip R.I.A. pentru web*. Bacău: Alma Mater, 2022.

[2] S. Buraga, *Tehnologii Web*. București: Matrix Rom, 2001.

[3] Mozilla, *MDN Web Docs – Using the Fetch API*. developer.mozilla.org

[4] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, teză de doctorat, University of California, Irvine, 2000.

[5] Open-Meteo, *Weather Forecast API – documentație*. open-meteo.com/en/docs

[6] C.-M. Papară, *Metodologie de lucru și template de proiect – Programare Web / Dezvoltarea Aplicațiilor Web*, UVAB, 2026.